SIH26037 — From Zero To The Project

Adaptive Path Planning & Collision Avoidance for Autonomous Vehicles on Unstructured Indian Roads · MathWorks · KIET team of 6

1. What we are making

A self-driving car that works on Indian roads — **entirely inside a computer simulation.** No real car, no hardware, nothing to buy. We build a virtual car, put it in a virtual Indian street, and show it driving through without crashing.

Five situations it must handle: an unmarked village road, a signal-less city intersection, a highway merge, a crowded market, and **a cow walking across the road.**

Our angle: every other team will build a car that *avoids* things. On an Indian road, a car that only gives way never moves. So we build a car that **negotiates** — it knows a cow ignores it, a pedestrian reacts to it, and a bus will push past it, and it behaves differently for each.

2. The journey — eight phases

```
PHASE 0 Get accounts and access            → we can log in everywhere
PHASE 1 Run someone else's example        → a car drives on our screen
PHASE 2 Build our own road                 → our village road + cow exists
PHASE 3 Add a normal planner               → the car we are going to beat
PHASE 4 Give the car eyes                  → our trained model runs inside it
PHASE 5 Add the negotiation                → our novelty is alive
PHASE 6 Measure and compare              → proof that ours is better
PHASE 7 Freeze, record, rehearse          → ready to present
```

Each phase ends with something visible. If you cannot see the thing at the end of a phase, we do not move to the next one.

3. The phases in detail

PHASE 0 — Get accounts and access (28–31 Aug)

START WITH: nothing.

DO:

- Create a **MathWorks account using your KIET email** at mathworks.com — this gives you MATLAB free through the college licence
- Register at **idd.insaan.iiit.ac.in** and download **IDD Lite (26.9 MB)** — small enough for any laptop
- Create a GitHub account and join our private repo
- Ask the lab in-charge about the supercomputer: which GPU, how to book it, how much storage, **how much free disk space**
- Shoot phone video of real Meerut roads — main road, gali, society lane, under-construction stretch

> **Done when:** everyone can open MATLAB, everyone has IDD Lite on their laptop, and we know what the supercomputer actually is.

PHASE 1 — Run someone else's example (1 Sept)

START WITH: MATLAB installed and nothing built.

DO: open one of MathWorks' own ready-made driving examples and run it completely unchanged. Do not modify anything. We are only proving the tools work on our machine before we build on top of them.

> **Done when:** a car drives across your screen in a simulation. It is not our car and not our road — that is fine. This is the moment the project becomes real.

PHASE 2 — Build our own road (2 Sept)

START WITH: a working example that isn't ours.

DO: run a script that creates **our** scenario — an unmarked village road, a few autos and bikes, and a cow that walks across at a set moment. Built to match the real road video we shot.

> **Done when:** the cattle-crossing scene exists and plays. The car drives straight and hits the cow — that is expected. We have not built the brain yet.

PHASE 3 — Add a normal planner (3 Sept)

START WITH: a road and a car with no brain.

DO: add a standard, ordinary path planner — the same kind every other team will use. It steers around obstacles and gives way.

This feels like wasted effort. It is not. This is the car we are going to beat, and without it we cannot prove our idea is better. Watch it carefully at the intersection: it will sit and wait for a gap that never comes.

> **Done when:** the car avoids the cow on its own — and freezes at the busy intersection. Both behaviours are what we want to see.

PHASE 4 — Give the car eyes (runs in parallel, 2–4 Sept)

START WITH: a car that is told where objects are.

DO: on the supercomputer, take the **YOLO** model and retrain it on Indian road photos from IDD. Export it as a single file, `best.onnx`. Copy that file to the MATLAB machine and load it into the simulation.

> **Done when:** the car identifies objects *itself* from the camera image — "that's an auto, that's a cow" — instead of being handed the answer.

PHASE 5 — Add the negotiation (4–5 Sept)

START WITH: a working but ordinary self-driving car.

DO: this is our actual contribution. Three things get added:

- **Negotiability per object** — a cow ignores us, a pedestrian reacts, a bus pushes past. Values measured from the IDD-X dataset.
- **Social priority** — right of way by size: truck > bus > car > auto > bike > pedestrian.
- **A new move: ASSERT** — the car can nudge forward to claim space instead of only waiting.

> **Done when:** the car that used to freeze at the intersection now negotiates its way through.

PHASE 6 — Measure and compare (5 Sept)

START WITH: two cars — the normal one and ours.

DO: run both through the same scenarios and log the three numbers MathWorks asks for: **replanning latency** (how fast it reacts), **path smoothness** (clean line vs jerky), and **scenario completion rate** (how many it finished without crashing).

> **Done when:** we have a side-by-side video — normal planner frozen on the left, ours negotiating through on the right — with real numbers under both.

PHASE 7 — Freeze, record, rehearse (6 Sept)

START WITH: something that works.

DO: **stop building.** Record the demo video. Copy everything to two laptops and a pen drive. Finish the PPT. Rehearse the demo at least five times, out loud, timed.

> **Done when:** the demo runs perfectly five times in a row, and there is a recorded video ready in case the live run misbehaves on the day.

Nothing new gets added on 6 September. One scenario that runs perfectly every single time beats four that are shaky. We have lost a hackathon before by demoing something with a bug we already knew about — a judge who hits that bug stops believing everything else we say.

4. Do we build MATLAB by hand, or does AI do it?

Short answer: **almost all of it is written by AI as code. You run it.** Nobody has to learn MATLAB properly or drag blocks around by hand.

Written as code by AI — you paste and run	Actually done by hand
All MATLAB scripts	Installing MATLAB
The Simulink model itself — built automatically by a script, not dragged together	Clicking Run
The road scenarios — roads, cars, the cow, their paths	Looking at what happened and describing it
Loading the trained model into MATLAB	Copying error messages back
The training code on the supercomputer	Screen-recording the demo
The measurement and logging code	Small visual tweaks if something looks wrong

So the working loop is:

```
Aditya asks the AI → AI writes the script → script goes into GitHub
    ↓
You pull it and run it → it works, or it errors
    ↓ if it errors
You copy the entire error message and send it back
    ↓
Fixed version comes back, usually in minutes → run again
```

> Copy the **whole** error, not a summary. "It didn't work" cannot be fixed. Twenty lines of red text can be fixed immediately. This one habit will save us days.

What you do need: two hours on the free MATLAB Onramp course at `matlabacademy.mathworks.com`. Not to learn MATLAB — just so that when the AI says "open the Configuration Parameters and change the solver," you know where to click.

5. Roles

These are a starting guess, not fixed. **One person can hold two roles, and two people can share one.** Sort it out among yourselves and change it whenever it stops making sense.

Role	Guess	What it covers
ML / Training	1	Supercomputer access, downloading the big datasets, running the training script, exporting `best.onnx`
Scripts / Integration	1	Runs everything the AI writes, reports errors back, keeps the GitHub repo tidy, connects the trained model to MATLAB
MATLAB & 3D world	1	Builds the road scenes and the simulation, assembles the Simulink model, produces the runs we record
PPT & documentation	2	Slides, technical report, the 90-second pitch, sources and citations, and rehearsed answers to hard judge questions
Demo & follow-up	1	Screen recording, editing the side-by-side video, backups, and chasing MathWorks about RoadRunner and their webinars

The load shifts through the week: training and datasets are heaviest early, MATLAB and simulation in the middle, slides and rehearsal at the end. Whoever is free helps whoever is stuck.

6. Quick reference

Accounts

Account	Where	Gives you
MathWorks	mathworks.com — **KIET email**	MATLAB free (college licence 41087767)
IDD	idd.insaan.iiit.ac.in	The Indian road datasets
GitHub	github.com	All our code

Datasets — and where they live

Dataset	Size	Goes on	Why
IDD Lite	26.9 MB	Your laptop	Get it today. Lets us test before the big files arrive.
IDD Detection	22.8 GB	Supercomputer	40,000 labelled Indian road photos. What we train on.
IDD-X	160 GB	Supercomputer	Where our negotiation values come from
IDD-3D	236 GB	Only if space allows	Skip for now

> **Big datasets never go on a laptop.** They stay on the supercomputer. Only one small file — the trained model — travels between machines.

The model we train

We do not build a neural network from scratch. We take **YOLO** (from Ultralytics), which is already trained on millions of general photos, and retrain it on Indian roads. That takes hours, not weeks, and it downloads itself the first time the training script runs.

The commands

On the supercomputer, once:

```
python -m venv sihenv
source sihenv/bin/activate
pip install ultralytics torch onnx
```

Then to train, and to export when it finishes:

```
python train_detector.py
python export_onnx.py
```

In MATLAB, to load the finished model:

```
net = importNetworkFromONNX("best.onnx");
save("detector.mat", "net");
```

What files go where

File	Lives on	Moves to
IDD datasets	Supercomputer	Nowhere — too big
All scripts	GitHub	Everyone pulls them
best.onnx	Supercomputer	**The one file that crosses over**, to the MATLAB machine
Simulink model (.slx)	GitHub	MATLAB machine
Demo video	Two laptops + pen drive	Backed up by 6 Sept

7. Dates

Date	What
31 August	Problem statement locked. Registration closes.
1–5 September	Phases 1 to 6.
6 September	**Stop building.** Record, back up, rehearse.
7 September	Internal hackathon. Present.